MANGASKETCH — TECHNICAL & DESIGN DOCUMENTATION

This document covers the complete system design, architecture, and technical decisions made during the planning and execution phases of MangaSketch.

PART 1: MY THINKING (PRE-BUILD DESIGN)

---

1. NICHE SELECTION & PRODUCT VISION

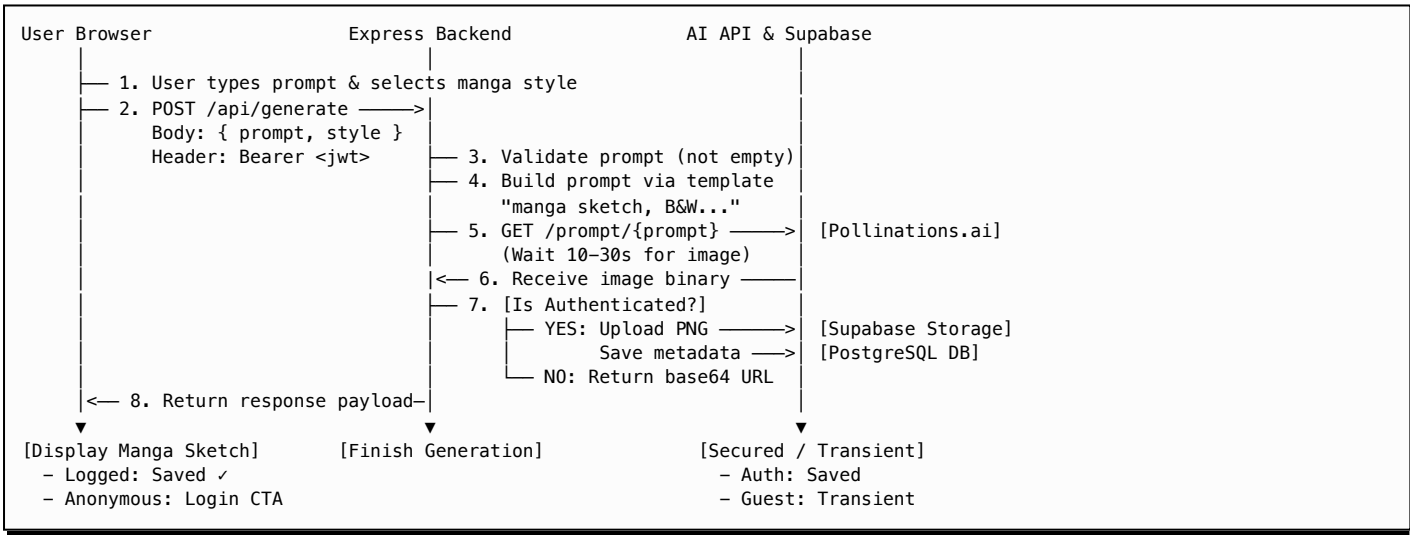
MangaSketch is built specifically for **mangakas (manga artists) and concept designers**. I chose manga as the niche because of my genuine passion and deep interest in manga, anime, and Japanese pop culture. This authentic connection inspired me to create a tool tailored precisely to the real-world workflow and aesthetic requirements of comic artists—ensuring the niche shapes the actual product design rather than just serving as a superficial label on top of a generic image generator.

- **The Problem:** Generic AI generators output high-contrast color illustrations that look nothing like traditional comic ink. When creating manga storyboards, character designs, or backgrounds, artists need black-and-white drawings, ink outlines, screentone dots, and clean lines.
- **The Solution:** MangaSketch wraps all user inputs in custom-tailored prompt engineering templates. It restricts output specifically to monochrome manga aesthetics, regardless of what the user types.
- **The Vision:** An AI concept artist assistant. Instead of separate, disjointed generations, each storyboard frame or sketch can evolve into versions (iterations) while preserving composition and seeds.

---

2. APP JOURNEY (DATA & REQUEST FLOW)

The diagram below illustrates our initial planning of how a user's prompt travels from the browser, through the Express backend, to the AI API, and back as an image:



---

3. TECH STACK JUSTIFICATION

LAYER	TECHNOLOGY	RATIONALE
Frontend	Next.js 15 (App Router) + TypeScript	Modern React framework with excellent optimization. TypeScript provides strict compile-time checks, ensuring type safety between backend responses and UI components.
Styling	Tailwind CSS v4	Provides rapid, responsive layout styling. The utility-first approach helps build complex neo-brutalist visuals easily.
Backend	Express + Node.js + TypeScript	Separating the backend ensures the Next.js client is unaffected by heavy image processing loads. TypeScript matches types with the frontend.
State Management	Zustand	Lightweight, selector-based state manager. Prevents heavy page-wide re-renders (unlike React Context) when sharing loading states and notifications.
Server State	TanStack Query (React Query)	Handles detail caches, timeline fetching, automatic invalidations, and smooth version switching without layout jumps.
Database & Auth	Supabase (PostgreSQL, Storage, Auth)	Full-featured managed PostgreSQL database with row-level security. Supabase Auth (Google OAuth) handles sign-in, and Storage handles secure asset hosting.
Image Processing	Sharp	High-performance Node.js image manipulation library. Used for Grayscale conversion and compositing SVG watermark buffers.

4. THE BUILD PROCESS (SEQUENCE)

We sequenced the planned development into 5 distinct phases to prioritize working software and validate core dependencies first:

- **Phase 1: Foundation (Day 1)**
  - Setup monorepo structure
  - Init Next.js + Express projects
  - Setup Supabase (database, storage, auth)
  - Test AI API (Pollinations), make sure it works before building anything on top
  - *Why first:* If the AI API doesn't work reliably, everything else is a waste of time. We must validate the core dependency first.
- **Phase 2: Core Backend (Day 2)**
  - Build `/api/generate` endpoint
  - Implement prompt wrapping (manga template)
  - Connect to Pollinations API
  - Implement image upload to Supabase Storage
  - Save metadata to PostgreSQL
  - Error handling for all failure states
  - *Why second:* Backend is the foundation. The frontend cannot do anything without a working API.
- **Phase 3: Core Frontend (Day 3-4)**
  - Build generate page with form
  - Implement loading experience (meaningful, not just spinner)
  - Display generated result
  - Handle all 3 error states visibly
  - Integrate Auth flow (Google login)
  - Build gallery page and detail page with re-generation
  - *Why third:* Once the backend is working, we can build the frontend with real data flowing through.

- **Phase 4: Polish + Deploy (Day 5)**
  - UI polish, make it feel like a real product, not an assignment
  - Deploy to Vercel + Railway
  - Test live URL end-to-end
  - Write setup instructions and documentation
- **Phase 5: Record Demo (Day 6)**
  - Record Loom demo highlighting the full journey (prompt, wait, image, save, re-generate)
  - Demonstrate at least two of the three required failure states on camera
  - Write down honest observations and system limitations

PART 2: MY DECISIONS (TECHNICAL CHOICES & EVOLUTION)

This section documents the actual engineering decisions made during development.

---

1. Renaming & Consistent Wording Overhaul

- **Original Plan (`thinking-old.md`):** Used terms like "gallery" (`/gallery`), "generations" (database table `generations`), "variants" and "drafts".
- **Actual Decision:** Refactored the entire project to use **"sketchbook"** (route `/sketches`), **"sketch(es)"** (database table `sketches`), and **"version(s)"** (instead of variants/drafts). All types in `@mangasketch/shared` and routes were strictly updated.
- **Why:** "Sketchbook" and "sketches" align much better with the creative manga niche, making the branding feel authentic. Consolidating terms prevents cognitive load and keeps codebase semantics clean.

2. Hanko Stamp Watermarking System

- **Original Plan:** No watermarking or signature features.
- **Actual Decision:**
  - Created a dynamic Japanese red Hanko stamp SVG (`HankoStamp.tsx`) showing Katakana text `マンガスケッチ` (MangaSketch) and optional user initials (max 4 characters).
  - Applied this stempel by default to all generated sketches using the backend `sharp` compositor.
  - Placed the Hanko logo behind the main brand text in `Header.tsx`.
- **Why:** The Hanko stamp acts as an authentic signature for manga artists. Putting it in the logo header creates visual continuity, linking the watermark stamp to the platform's brand identity.

3. Grayscale Processing via Sharp

- **Original Plan:** Rely on prompt engineering (e.g., *"black and white, manga panel"*) to get monochrome images.
- **Actual Decision:** Programmatically converted the raw image buffer to grayscale using `sharp(buffer).grayscale().toBuffer()` in the backend before adding the watermark.
- **Why:** Diffusion models (like Pollinations) are stochastic and often leak sepia, blue, or yellow tones even with strict B&W prompts. Forcing grayscale in the backend guarantees a pure monochrome output, ensuring a consistent manga-paper aesthetic.

4. Overhauled Watermark Style Selector

- **Original Plan:** Standard textual style chips.
- **Actual Decision:**
  - Built interactive grid selector buttons with transparent manga drawings as backgrounds.
  - Created a Python script to crop out black borders from raw AI images and mask grayscales into pure transparent ink PNGs (`alpha = 255 - gray`).
  - Converted the final background images to WebP format, resized them from 1024x1024 down to 256x256, and optimized compression with 80% quality.
  - Added theme-aware CSS image filters to adapt the selector backgrounds dynamically to Light, Tankobon (sepia), and Midnight (dark) themes.
- **Why:** Provides visual examples for style selections (Shonen, Seinen, Shojo, Chibi, etc.). Transparent alpha masking ensures background images adapt cleanly to the active theme without losing contrast or readability. The WebP conversion and resolution reduction slashed the total asset weight by 95% (from ~6.5MB to just ~310KB), eliminating initial loading lag on the style pills.

5. Post-Authentication Cache Recovery (UX Preservation)

- **Original Plan:** Guest users click login, losing any prompt they had typed.
- **Actual Decision:**
  - If a guest generates a sketch, the app caches the prompt, styles, seed, and base64 image URL in `localStorage` (`mangasketch_pending_upload`).
  - After logging in and redirecting to `/auth/callback`, the app reads this cache and calls a dedicated backend route handler to upload the base64 image directly to Supabase storage and persist the metadata.
- **Why:** Prevents users from losing their prompts and generated artwork during OAuth redirects. The sketch is recovered and saved to their sketchbook automatically, providing a seamless user experience.

6. Split-Server Entry Point Refactoring

- **Original Plan:** Single `server.ts` entry point binding to ports.
- **Actual Decision:** Split the Express backend into `app.ts` (defining routes, middleware, and Express configuration) and `index.ts` (binding to the network port).
- **Why:** In integration testing (Vitest + Supertest), the test runner needs to mount the Express app without listening on a physical port. Splitting these files prevents "address already in use" errors during parallel test execution.

7. Snappy Deletion & Asynchronous Invalidation Transitions

- **Original Plan:** Standard page refreshes or blocking waits during deletions.
- **Actual Decision:**
  - Refactored the detail page deletion to instantly close the modal and update the UI router state (navigating to the latest remaining version) *before* invalidating queries in the background (no `await`).
  - Reduced the deletion toast autohide duration in the Zustand store from 4000ms to 2500ms.
- **Why:** Keeps the UI responsive. Waiting for the database invalidation to resolve blocked page transitions. Running it asynchronously makes transitions instant, and a shorter toast duration matches the quick deletion flow.

8. Smart Input Change Detection

- **Original Plan:** The submit button is always active.
- **Actual Decision:**
  - Added a change listener comparing the current form state to the active sketch's parameters.
  - Displays a red `[ CHANGED ]` badge next to modified inputs.
  - Changes the submit button text from `NO CHANGES DETECTED` (disabled) to `SKETCH NEW VERSION` (enabled).
- **Why:** Prevents duplicate generations of identical sketches, saving API limits. It clearly informs users whether their current configuration will fork a new version or matches the loaded one.

9. Speech-Bubble Comic Tooltips

- **Original Plan:** Standard browser `title` tooltips or truncated text.
- **Actual Decision:** Created a custom monospace tooltip component styled like a manga speech bubble with thick borders, a flat box shape, and a tiny directional triangular notch.
- **Why:** Reinforces the manga theme. Replacing native browser tooltips with a thematic speech bubble makes viewing long prompts fun and matches the overall aesthetic.

10. Anti-Save & Drag Protection

- **Original Plan:** Standard HTML `<img>` elements.
- **Actual Decision:** Disabled right-clicks (`onContextMenu={(e) => e.preventDefault()}`) and image dragging (`draggable={false}`) on all canvases, overlays, and sketchbook thumbnails.
- **Why:** Protects platform artwork and simulates a premium environment. It encourages users to use the official "Download Panel" button, which applies the Hanko stamp watermark.

11. Unified Delete Confirmation Modal with Card Preview

- **Original Plan:** A generic text warning dialog.
- **Actual Decision:** Developed a unified `<DeleteConfirmationModal />` displaying a horizontal mini-card preview of the target sketch (portrait thumbnail on left, metadata + prompt on right, and a badge identifying if it is the original sketch or a child version).
- **Why:** Deleting an original sketch cascades and erases all child versions. Providing a detailed visual preview card prevents accidental deletion of entire sketch families.

12. Flat-Tree DB Versioning

- **Original Plan:** A nested tree hierarchy.
- **Actual Decision:** Flat-tree database design where all variations point directly to the original root sketch (`parent_id = root parent id`).
- **Why:** Avoids complex, recursive PostgreSQL queries (Common Table Expressions) that degrade query performance as version history grows. This flat-tree structure makes retrieving a sketch's entire version history fast and scalable.

13. Zustand for Global UI State

- **Original Plan:** React Context for sharing state.
- **Actual Decision:** Created a Zustand store (`useUiStore`) to manage loading, toast alerts, and navigation indicators.
- **Why:** Context triggers a full re-render of child components whenever any value changes. Zustand's selector-based subscription prevents layout updates, keeping page renders lightweight.

14. Retro 3-Mode Theme Switcher

- **Original Plan:** Standard Light/Dark mode.
- **Actual Decision:** Created three customized themes:
  - **Light Ink:** High-contrast white paper.
  - **Recycled Book / Tankobon:** Warm sepia paper color resembling vintage manga magazine print.
  - **Midnight Moon:** High-contrast dark mode.
- **Why:** Reduces eye strain during night reading. The Tankobon theme offers a nostalgic feel that matches cheap newsprint manga magazines like *Weekly Shōnen Jump*.

15. Backend-to-Frontend WebP Pipeline Migration

- **Original Plan:** Save and serve files in standard raw PNG/JPEG formats.
- **Actual Decision:** Migrated the backend watermark composition output from PNG to WebP with 85% quality, changing file extensions in Supabase storage and API response payloads.
- **Why:** Grayscale/monochrome manga line art has large blocks of solid black and white. This matches WebP's compression algorithm perfectly, slashing file sizes by up to 90% (from ~2.0MB to ~100-150KB) while keeping lines crisp. This makes versions, sketchbook galleries, and canvases load instantly.

16. Client-Side High-Res PNG Download Reconstruction

- **Original Plan:** Trigger direct browser download of the image URL served from storage.
- **Actual Decision:** Overhauled the download button to fetch the WebP file, load it into an offscreen image, draw it on an offscreen `<canvas>` at its exact natural resolution, and export it back to a high-compatibility PNG file via `canvas.toBlob(..., 'image/png')` before triggering the browser's download prompt.
- **Why:** Keeps the web interface lightning-fast by transferring lightweight WebP files, but preserves high-compatibility HD PNG output for users importing sketches into legacy graphic design tools (such as Photoshop, Procreate, or Clip Studio Paint) that do not natively open WebP files.

17. Daily Reset Calendar-Day Rate Limiting (Ink Quota)

- **Original Plan:** Standard sliding-window rate limiters.
- **Actual Decision:** Implemented a calendar-day rate limiter resetting at exactly `00:00 UTC` daily, with guest limits (5 generations/24h) and authenticated limits (15 generations/24h).
- **Why:** Sliding-window rate limiters require keeping sliding timestamp arrays in memory per key, raising system memory costs. Calendar-day resets simplify arithmetic (using `getNextMidnightUTC()`), decrease overhead, and offer a transparent, predictable schedule to the user.

18. Hanko Stamp Vector Outlines & Initials Validation

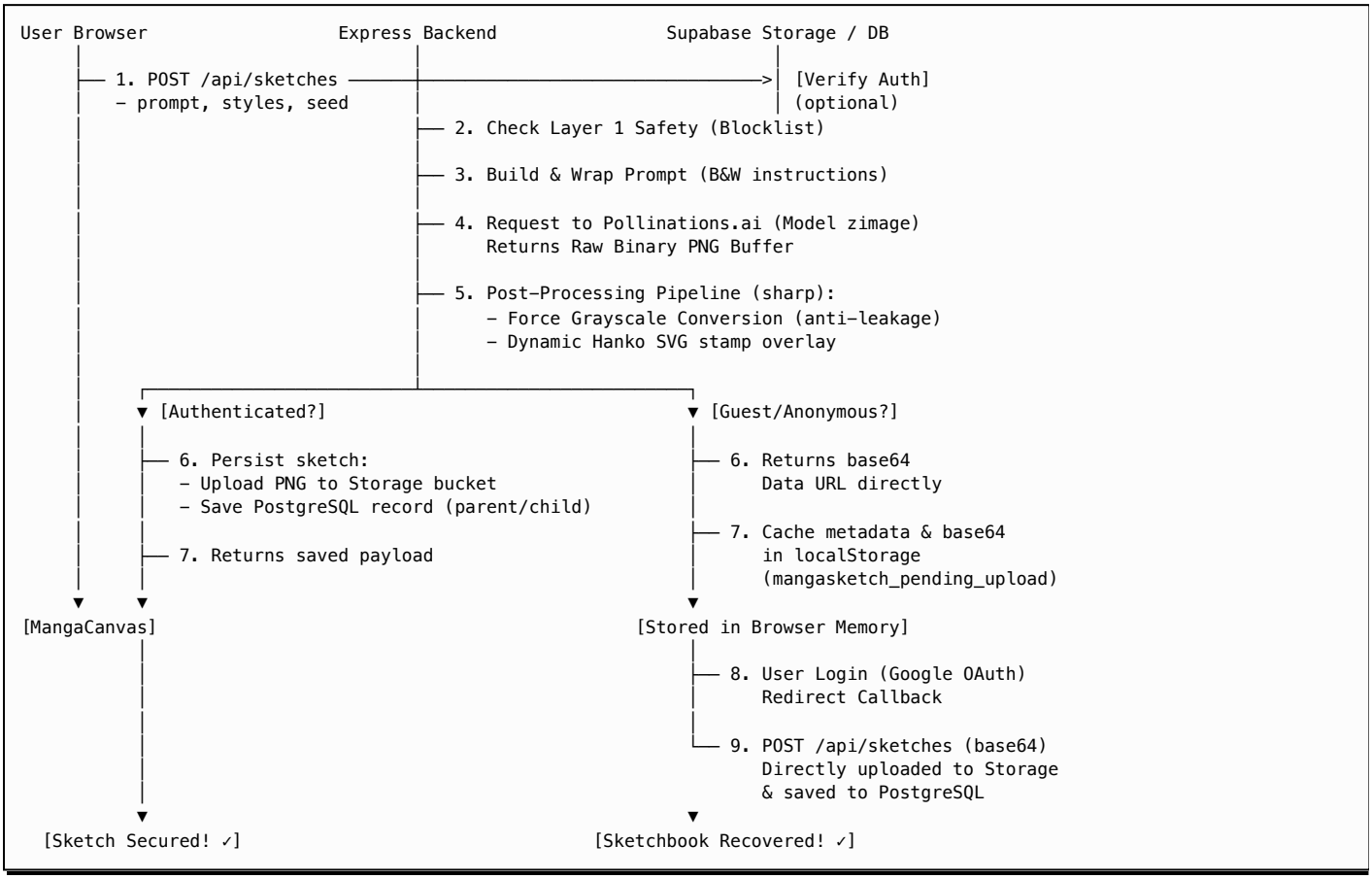
- **Original Plan:** Rely on system-level font installations (e.g., expecting *Noto Sans JP* and *Impact* to be pre-installed on the host operating system) or inline Base64 WOFF2 font embedding inside the SVG.
- **Actual Decision:** I converted the entire backend Hanko stamp watermark from font-dependent SVG text elements into 100% vector-based paths. For the Japanese characters, I pre-extracted static vector outlines for `マ`, `ン`, `ガ`, `ス`, `ケ`, `ッ`, `チ` (stored in `katakanaPaths.ts`). For the user initials, I integrated `opentype.js` to dynamically parse `impact.ttf` at startup and convert the letters into vector paths (`d` path coordinates) positioned precisely using character advance widths. I also configured the build system to copy `impact.ttf` to `dist/` and tightened initials validation: the frontend strips spaces and illegal characters in real-time, while the backend ensures the input strictly contains only alphanumeric characters.
- **Why:** During deployment on Railway, I discovered that headless Linux container environments do not have Japanese fonts installed, and Sharp's underlying SVG renderer (`librsvg`) does not support parsing Base64-encoded fonts inside `@font-face` rules. This caused text watermarks to render as broken empty boxes (tofu). Converting the characters to vector paths makes the watermark completely independent of system fonts or CSS font loading, ensuring it renders identically and consistently across any server setup. Disallowing spaces in initials prevents unsupported character layouts from breaking the stamp's visual integrity.

ACTUAL APP JOURNEYS & OPERATIONS (HIGH-FIDELITY FLOWS)

These diagrams visualize the finalized actual architectures and request lifecycles implemented in the production codebase.

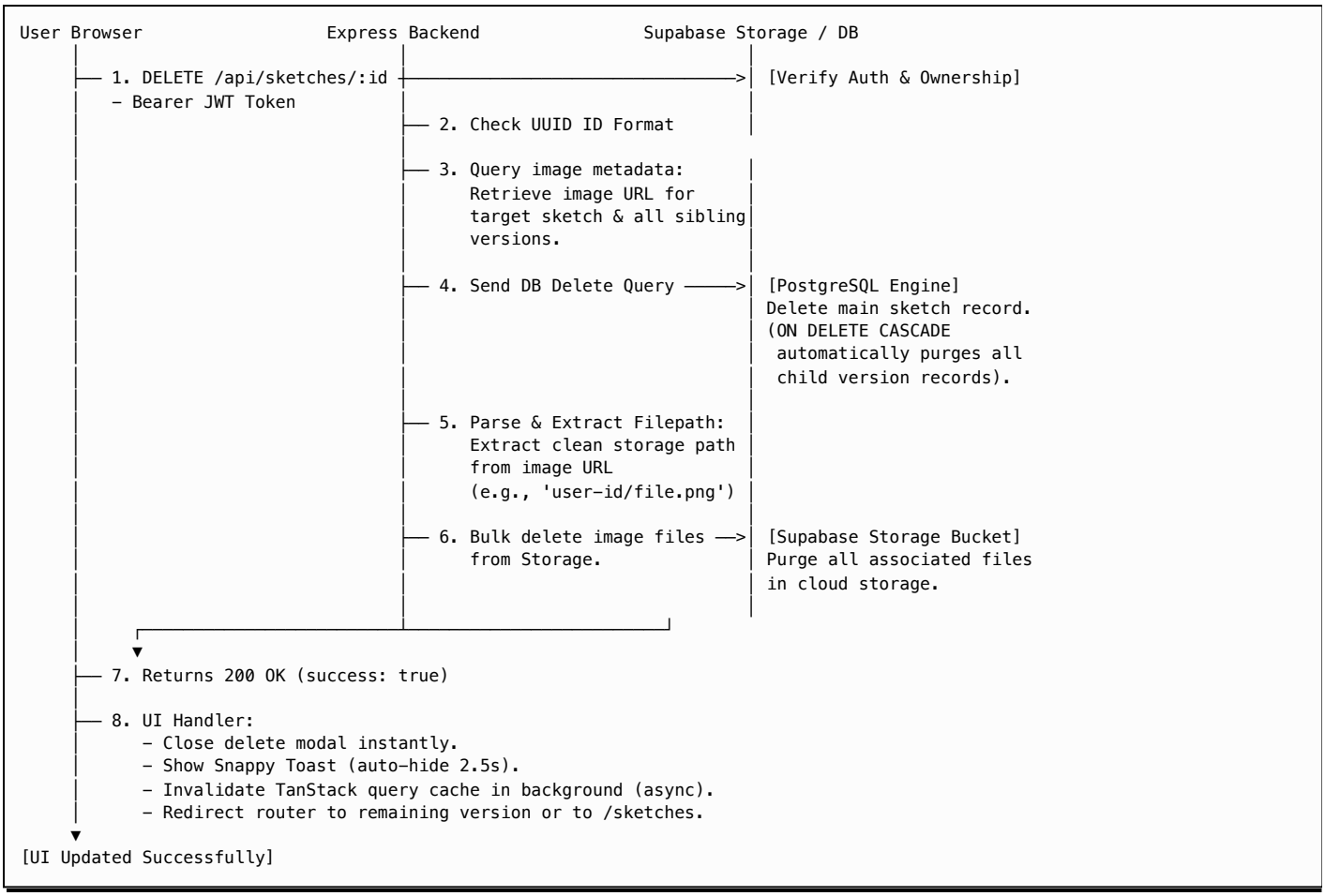
A. Actual Generation Journey (POST /api/sketches)

This flowchart illustrates the end-to-end request loop for generating a new panel, including the backend sharp processing pipeline and the localStorage guest cache recovery flow:



B. Actual Deletion & Cleanup Journey (DELETE /api/sketches/:id)

This flowchart illustrates how a sketch or variation is permanently purged from the workspace. It details the cascading database purge and the bulk file deletion from cloud storage:



OVERLY DEVELOPED & POLISHED FEATURES

The assignment brief says this project should take 7 to 8 hours of focused work. But since I have a 7-day deadline, I choose to spend a few hours each day to chip away at it according to my initial plan, plus making some adjustments along the way. I really fall in love with the manga niche and this project, so I decided to build it like a real product with high attention to product flow, UI, and UX polish.

On the backend side, I keep it as simple and minimal as possible because I am a frontend-heavy fullstack engineer. But for the product experience and UI/UX, I go far beyond the requirements to make this a stellar piece in my personal portfolio.

Here are the features I built that are not in the requirements:

1. Google OAuth Auth & Guest-to-User Conversion Flow

- **Why built:** I want to separate the anonymous guest flow and the registered user flow. It works like a real SaaS product: guests can generate 5 sketches/day (tracked by IP) to test the app, but they see a CTA banner to login. If they log in via Google, their daily quota increases to 15, and all their drawings are saved permanently to PostgreSQL and Supabase Storage instead of disappearing. This makes data management much easier and creates a natural user acquisition loop.

2. Sketches & Version Deletion Flow (Full CRUD)

- **Why built:** The brief only asks to save and show sketches. But I want to make the CRUD cycle complete. Also, artists in real life often make rough drafts they do not like and want to trash them. Without a delete feature, the sketchbook gallery will be full of garbage sketches, which ruins the long-term user experience. I built a unified neobrutalist `DeleteConfirmationModal` that handles deleting specific versions (V2/V3) or scrapping the whole version family (V1).

3. Manga Niche Multi-Theme System (Born from Personal Experience)

- **Why built:** I got this idea from my own experience while developing this project. Because I often code late at night in a dark room, the stark white background (which is the main color of manga art) was really hurting my eyes. So I built a theme switcher with 3 options: Light, Tankobon (warm cream color like real physical manga paper), and Midnight (neo-brutalist dark mode) with theme-aware image filters. It makes the site much more comfortable to use.

4. Chronological Version Control History (Git for Sketches)

- **Why built:** In a real manga drawing workflow, artists iterate their sketches step-by-step. Re-generating shouldn't just overwrite the old drawing or save them as completely separate items in the gallery. I built a version tree system (V1, V2, V3...) with a horizontal history timeline and zero-latency caching using TanStack Query. Simple navigation transitions allow artists to switch between versions instantly to see their creative progress.

5. Hanko Stamp & Manga Signature Marriage

- **Why built:** I wanted to make a fun signature feature that stands out. In Japan, a Hanko is a personal name stamp used for official documents (like opening bank accounts or signing contracts) instead of a handwritten signature. It is not normally used on manga panels (mangakas usually use a handwritten sketch signature or street sign). But I decided to marry the Hanko stamp concept with the manga panel signature! The red Hanko stamp with Katakana マンガスケッチ (MangaSketch) acts as a branding watermark showing the image was generated on this platform, and users can add their own initials (up to 4 characters) to the bottom of the stamp to make it their own personal Hanko signature, just like in real life.

6. Server-Side Grayscale Processing (Sharp)

- **Why built:** Text-to-image AI models are random. Even if we write "black and white manga sketch", they often leak sepia, yellow, or blue tones. Programmatically converting the image buffer to grayscale in the backend before saving guarantees a 100% consistent traditional manga paper look.

7. WebP-to-PNG CORS-Safe Download & Anti-Save Protection

- **Why built:** I want the website to load super fast, so the backend saves images in WebP format (saving 90% file size, from 2.6MB to 270KB). But artists need high-quality PNGs for their portfolios. I built a custom download utility that programmatically draws the WebP onto an offscreen canvas and exports it as a high-res PNG, while disabling standard browser right-click dragging to protect the artwork.

8. Monospace Manga Speech-Bubble Tooltips

- **Why built:** Instead of using boring default browser tooltips for long prompts, I styled absolute-positioned tooltips to look exactly like manga speech bubbles (monospaced font, thick flat borders, and a tiny pointing notch) to keep the comic book theme consistent down to the micro-details.

9. Fully Automated Backend Test Suite (Vitest & Supertest) with Concurrency and Rate Limit Validation

- **Why built:** The assignment specifies that the app must work correctly with multiple users generating at the same time and handle complex "messy" cases like concurrent load and rate limits. Rather than relying on manual browser clicking to guess if the concurrency works, I wrote a comprehensive automated testing suite (comprising 29 test cases) using **Vitest** and **Supertest** to mathematically prove the backend's robustness. The test suite mocks out slow network boundaries (like Pollinations.ai and Supabase Auth) to execute instantly, and asserts:
  - **Thread-Safety & Concurrent Load**: Verifies that when multiple parallel/concurrent requests are fired, the server handles state changes and database actions safely without race conditions or memory leaks.
  - **Quota Limits**: Simulates and asserts that anonymous guests are strictly capped at 5 generations/day and authenticated users at 15 generations/day, checking that the rate limit middleware properly returns `429 Too Many Requests` when limits are breached.
  - **Input Pre-flight Rules**: Validates that prompt injections or invalid requests are blocked before calling the AI generation layer.
  - **Asset Verification**: Checks that grayscale filters are correctly applied and watermark coordinates generate accurately.

KNOWN LIMITATIONS & PRODUCTION TRADE-OFFS (THE HONEST PART)

This section documents the current limitations of the implementation and outlines proposed production-grade solutions for future scalability.

1. Unpaginated Sketchbook Fetching

- **Current Limitation:** The `GET /api/sketches` endpoint fetches all sketches belonging to the user in a single database query to simplify frontend state synchronization and TanStack Query cache invalidations.
- **Impact:** As a user's sketchbook grows to hundreds or thousands of sketches, this will increase payload sizes, payload transmission times, and frontend grid rendering times.
- **Production-Grade Solution:** Implement cursor-based pagination (infinite scroll) in the API and UI to fetch sketches in chunks (e.g., 20 sketches per load).

2. Missing Database Watermark Attributes

- **Current Limitation:** The database schema for the `sketches` table does not persist `watermark_text` or `watermark_position` values.
- **Impact:** When loading a sketch in the detail page workspace, the re-sketch form resets to default watermark parameters (empty name, `BOTTOM_RIGHT` position) instead of pre-filling the actual parameters used in that specific version (though the saved image file itself remains watermarked).
- **Production-Grade Solution:** Add `watermark_text` (nullable) and `watermark_position` columns to the PostgreSQL schema to persist and return these values with sketch metadata.

3. Stochastic Nature of Diffusion Models (Seed Drift)

- **Current Limitation:** Even when locking the generation seed, modifying text prompt tokens can still trigger visual shifts or composition drift in the character structures.
- **Impact:** Generations are stochastic by nature, meaning visual consistency cannot be 100% guaranteed across versions.
- **Production-Grade Solution:** Integrate ControlNet line-art or Canny edge conditioning to use the previous version's image coordinates as a physical boundary.

4. Pollinations.ai Model Style Adherence Limitations

- **Current Limitation:** Even though we have crafted highly optimized, distinct prompt modifiers for each Manga Style (Shonen, Seinen, Shojo, Chibi) and Drawing Style (Rough Sketch, Clean Line Art, Inked Manga, Illustration) in `apps/server/src/utils/promptHelper.ts`, the default free models used (`flux` or `zimage`) show limited adherence to these specific artistic style changes. While `zimage`/`flux` can capture distinct manga styles reasonably well, they struggle to differentiate between drawing styles (e.g., distinguishing between a "Rough Sketch" with pencil construction lines and "Clean Line Art"). We experimented with more advanced models like `klein`, but due to high token/pollen consumption, we reverted to the more cost-effective `zimage`/`flux`.
- **Impact:** The visual styles of generated panels might look somewhat similar or fail to fully reflect the selected drawing style (e.g., a "Rough Sketch" might look too clean, or "Illustration" might lack sufficient detail).
- **Production-Grade Solution:** Integrate premium paid APIs (such as Midjourney, Stable Diffusion 3, or customized DALL-E 3 endpoints) that offer superior prompt-adherence, or train a custom LoRA model specifically on manga drawing styles (Rough, Inked, Clean, etc.) and deploy it to a dedicated GPU host.

5. Third-Party API Dependency (No Uptime SLA)

- **Current Limitation:** Using the free tier of Pollinations.ai has no service-level agreement (SLA) or guaranteed response times.
- **Impact:** If the provider experiences traffic overload or server downtime, sketch generation will fail or timeout.
- **Production-Grade Solution:** Setup a multi-provider fallback system (e.g., automatically switching to Gemini Flash or Replicate if the primary provider fails).

6. In-Memory Rate Limiting

- **Current Limitation:** The anonymous rate limiting (max 5 sketches/hour) is stored in the Node.js backend's runtime memory.
- **Impact:** If the server container restarts or redeploys on Railway, all user generation counters reset.
- **Production-Grade Solution:** Use a persistent distributed key-value store like Redis to manage rate-limit keys.

7. LocalStorage Size Constraints for Guest Cache

- **Current Limitation:** Anonymous sketches are cached as raw base64 data URLs in `localStorage` to preserve them across login redirects.
- **Impact:** Browser `localStorage` is restricted to ~5MB. Since a single base64 PNG sketch can consume 600KB - 1MB, guest users will hit browser storage limits if they try to save multiple temporary sketches before logging in.
- **Production-Grade Solution:** Upload guest sketches to a temporary server storage folder and save only the temp ID in browser cookies.

8. Orphaned File Risks on Database Transactions

- **Current Limitation:** Deleting a sketch deletes the database records first and then triggers bulk file deletion in Supabase Storage.
- **Impact:** If a network failure occurs after the database transaction commits but before the storage deletion resolves, physical files are orphaned in the cloud bucket.
- **Production-Grade Solution:** Use distributed transaction patterns (saga pattern) or a background cron job that periodically audits and sweeps storage files without matching database records.

9. Decoupled Authentication Failure Mode

- **Current Limitation:** Authentication is handled directly on the frontend using the Supabase Client SDK, while the custom Express backend only decodes and validates the Bearer JWT token in middleware.
- **Impact:** If the Express backend server is offline or experiencing database connection errors, the frontend website still loads and the user can successfully sign in/out via Google OAuth (since Auth requests go directly through Supabase API servers). However, the logged-in user will be unable to generate sketches, view their sketchbook, or perform deletions, leading to a confusing UX where "login succeeds, but the application is non-functional."
- **Production-Grade Solution:** Route all auth actions through backend session endpoints (e.g., Express-managed session cookies) or implement a client-side health-check ping to the Express server to show a "Server Offline" banner and disable interaction if the backend is unreachable.

10. Storage CORS Dependency for Clientside PNG Reconstruction

- **Current Limitation:** The frontend canvas-based WebP-to-PNG download reconstruction relies on fetching the image buffer via `fetch(imageUrl)` from Supabase Storage.
- **Impact:** If the Supabase Storage bucket's CORS policy is misconfigured or lacks wildcard origin headers (`Access-Control-Allow-Origin: *`), the browser blocks the fetch request. The user is then forced to download the sketch via a fallback new-tab redirection, which serves the raw WebP file instead of the requested PNG.
- **Production-Grade Solution:** Route image downloads through a custom proxy endpoint on the backend (e.g., `GET /api/sketches/:id/download`) which pulls the buffer server-side and streams it to the client with `Content-Disposition: attachment` headers, bypassing clientside CORS checks entirely.

11. Flat Database Schema for Hierarchical Sketch Families

- **Current Limitation:** The database only has one table called `sketches` to store everything. Because there are no separate tables for "Sketch Families" (projects) and "Versions" (drawings), the version tree hierarchy feels a bit messy. The data we save is also very minimal—we don't have user-friendly sketch names, only long UUID strings.
- **Impact:** The frontend has to maximize the display using only this minimal data (e.g., cutting the long UUID into short titles like `PANEL #83735fdc` and calculating version numbers like `V1` or `V2` on the fly by sorting dates in memory). This makes the sketchbook flow feel a bit weird because we don't have proper project names or custom labels.
- **Production-Grade Solution:** Create a normalized database schema with a `sketch_families` table (to store project names, custom labels, description, and user_id) and a separate `sketch_versions` table (to store image_url, prompt, seed, styles, and parent_id relationship) linked via foreign keys.